Parse, Don't Validate

✦ When to apply: boundaries & IO

Enchantment · Beginner · TypeScript

✕ Parse Once · Type-Safe · ✦ Beginner

Transform untyped data into typed structures at system boundaries. Once parsed, trust the types everywhere inside your application.

```
const user = ZodUserSchema.parse(json)

// user.name & user.email guaranteed
// valid everywhere — no re-checking
```

"One badge swipe and you're in the club. No repeated interrogations at every door inside."

Learn more

Shotgun Parsing

☠ ANTI-PATTERN OF: PARSE, DON'T VALIDATE

CURSE | SCATTERED IO | AVOID

💀 FRAGILITY: MAX ⚠ SPREAD: 5 FILES ☠ AVOID

Validation logic scattered throughout the codebase. Each function checks only the fields it needs — one unchecked field anywhere crashes everything.

```
if (json.name &&
typeof json.name !== 'string')
throw new Error('invalid name')
// repeated in 5 files. json.email? 💥
```

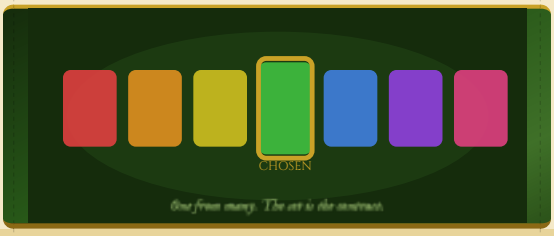
"Walking on thin ice — every single step is terrifying. One wrong crack and the whole program falls through."

JESSEWARDEN.COM

Discriminated Unions

✦ WHEN TO APPLY: DOMAIN MODELING, STATE MACHINES

ENCHANTMENT | BEGINNER | ⚡ MOST IMPORTANT



Tagged Variants | 🔀 Exhaustive | ✦ Beginner

Use a *tag* field to distinguish between variants of a type. Enables exhaustive pattern matching — the most important card in the deck.

```
type Result<T,E> =
| { tag: 'Ok', value: T }
| { tag: 'Err', error: E }
```

"A row of weapons — you can carry all, but only draw one. The set defines your world."

LEARN MORE

Boolean Blindness

ANTI-PATTERN OF: DISCRIMINATED UNIONS

CURSE | LOST CONTEXT | AVOID

Lost Context | Runtime Checks | Avoid

Using booleans or strings to represent states loses information and requires runtime checks to determine meaning.

```
process(true, {}, undefined)
// caller must remember what true means
// often must read implementation 😬
```

"Did you ship my package?" "No." – waiting? in customs? lost? 'No' tells you nothing."

JESSEWARDEN.COM

Make Impossible States Impossible

◆ When to apply: domain modeling, state machines

Enchantment · Beginner

🔒 Type-Locked ✅ No Invalid Path ✦ Beginner

Design types so invalid states cannot be represented. Use discriminated unions to make each state explicit and compiler-enforced.

```
type Request =
| { tag: 'loading' }
| { tag: 'success', data: User }
| { tag: 'error', error: string }
```

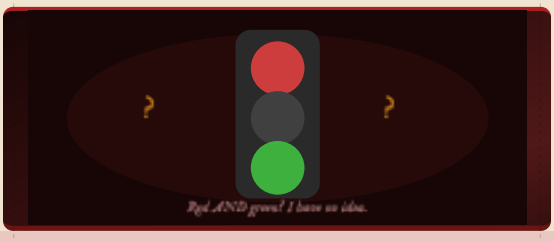
"Pro athletes visualize success, then go do it. They can't screw up if success is the only path."

Learn more

Invalid State Combinations

☠ ANTI-PATTERN OF: MAKE IMPOSSIBLE STATES IMPOSSIBLE

CURSE | OPTIONAL FIELDS | AVOID



💀 Contradictory ⚠ Optional Fields ☠ Avoid

Optional fields with implicit relationships — certain combinations are meaningless or contradictory. What does loading:true with data AND error mean?

```
type Request = {
loading: boolean
data?: User
error?: string
} // 8 states, most are nonsense
```

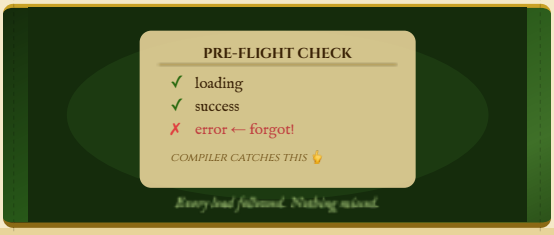
"Red AND green at the same time. I'd try to go... but only if others were stopped everyhw?"

JESSEWARDEN.COM

Exhaustiveness Checking

◆ WHEN TO APPLY: DOMAIN MODELING, STATE MACHINES

ENCHANTMENT | BEGINNER



🔍 Compile-Time ✅ All Cases ✦ Beginner

Compiler ensures all union cases are handled. Add a new variant and the compiler shows every place that needs updating.

```
switch(request.tag) {
case 'loading': ...
case 'success': ...
// compiler: forgot 'error'! ✗
```

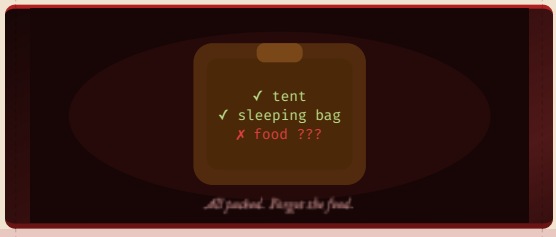
"If you're a detective, every possible lead is followed. No forgotten case bugs."

LEARN MORE

Forgotten Case Bugs

ANTI-PATTERN OF: EXHAUSTIVENESS CHECKING

CURSE | RUNTIME ONLY | AVOID



RUNTIME SURPRISE ⚠ STRING STATUS AVOID

Adding a new string status and missing handlers in switch statements — discovered only at runtime, never at compile time.

```
switch (request.status) {
case 'loading': ...
case 'success': ...
// added 'cancelled' — oops 💥
```

"Left the house with wallet, phone, laptop. Forgot the keys."

JESSEWARDEN.COM

Result Types

✦ When to apply: boundaries/IO, API design

Enchantment · Beginner

🎯 Explicit Failure 🔀 Forced Handle ✦ Beginner

Return success or failure explicitly in the type. Caller must handle both cases — failure is a visible possibility, not a hidden surprise.

```
function readFile(fileName: string):
Result<File, NotFound | NoPermission>
```

"The chef is out of ramen — but recommends katsu curry because she knows you like pork."

Learn more

Exception Blindness

ANTI-PATTERN OF: RESULT TYPES

CURSE | HIDDEN FAILURE | AVOID

💀 Hidden Throws ⚠ Bypass Types ☠ Avoid

try/catch doesn't tell you what can fail from the type signature. Exceptions bypass the type system entirely.

```
function getUser(id: string): User
// throws NotFoundError?
// throws ValidationError?
// throws DatabaseError? Who knows! 🎲
```

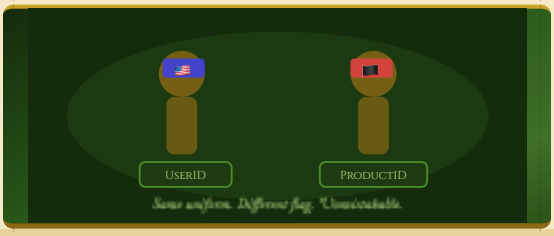
"I have a problem" — then silence. Do they need help now? No hint at what to try."

JESSEWARDEN.COM

Branded Types

✦ When to apply: Domain modeling

Enchantment · Intermediate



Semantic Tags · Compile-Time · Intermediate

Distinguish semantically different values that share the same primitive type. The compiler prevents mixing them up.

```
type UserID = string & { brand: 'UserID' }
type ProductID = string & { brand: 'ProductID' }
// mixing them → compile error ✓
```

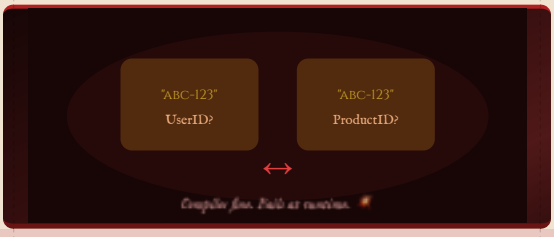
"Soldiers tagged with a country flag — uniforms look similar, but the brand makes them unmistakable."

Learn more

Primitive Obsession

☠ ANTI-PATTERN OF: BRANDED TYPES

CURSE | ID MIXUP | AVOID



💀 ID Confusion ⚠ Silent Mixup ☠ Avoid

Accidentally passing one ID where another is expected. The compiler can't distinguish between two string values — they look identical.

```
getProduct(userID)
// compiles fine — both are string
// fails at runtime 💥
```

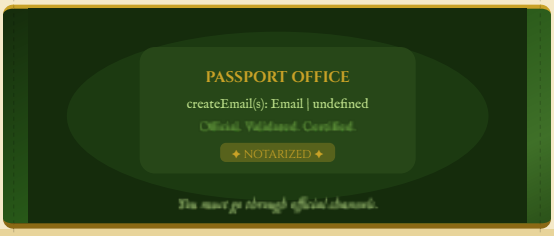
"I thought this was a bell pepper... it was a habanero. Looked identical from outside."

JESSEWARDEN.COM

Opaque Types

✦ WHEN TO APPLY: DOMAIN MODELING, MODULE-LEVEL

ENCHANTMENT | INTERMEDIATE



🏛️ Factory Only 🔒 Hidden Internal ⚡ Intermediate

Hide internal representation, force construction through a validated factory function. The type is only constructable one way.

```
type ValidEmail = ... // internal only
// only exported:
createEmail(s: string): ValidEmail | undefined
```

"Go to the passport office. You'll get your passport. No shortcuts — that's the guarantee."

LEARN MORE

Accidental Forgery

ANTI-PATTERN OF: OPAQUE TYPES

CURSE · FAKE VALUES · AVOID

ALI EXPRESS NINJA 400

$2,000 — too good to be true

⚠ COUNTERFEIT ⚠

Looks legit. Compiles fine. Disaster inside.

💀 Fake Values ⚠ Bypass Validate ☠ Avoid

Creating invalid values directly by bypassing validation — since the type is just a primitive alias, anything compiles.

```
const email: Email = "not-an-email"
// compiles if Email = type Email = string
// disaster at runtime 💥
```

"Saw a Kawasaki Ninja 400 on AliExpress for $2k. The pictures were convincing. No way that was real."

JESSEWARDEN.COM

NonEmpty Collections

✦ WHEN TO APPLY: DOMAIN MODELING

ENCHANTMENT · INTERMEDIATE

GROCERY LIST

✓ wine (1 bottle)
✓ weird gift (1)
∴ always ≥ 1 item

"You don't need to bring anything." 99% do.

📦 At Least 1 · 🛡️ No Empty · ⚡ Intermediate

Type that guarantees at least one element exists in an Array. No more runtime checks for potentially empty arrays.

```
type NonEmptyArray<T> = [T, ...T[]]
// or
type AtLeast1<T> = { first: T, rest: T[] }
```

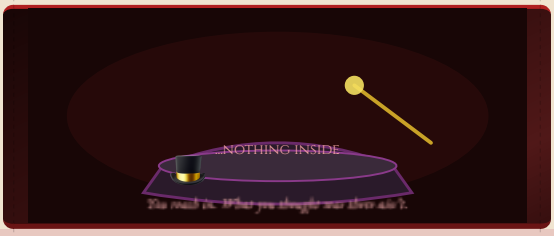
"Her Majesty sends me to the store. I always buy something anyway — empty list is not an option."

LEARN MORE

Empty Collection Crashes

☠ ANTI-PATTERN OF: NONEMPTY COLLECTIONS

CURSE | UNDEFINED[0] | AVOID



💀 UNDEFINED[0] ⚠ REDUCE THROWS ☠ AVOID

Accessing first element or reducing without initial value on potentially empty arrays — silently undefined or a thrown error.

```
const first = items[0]
// undefined if empty ☠
items.reduce((a,b) ⇒ a + b)
// throws if empty 💥
```

"You reach in the hat expecting a rabbit. What you thought was there ain't."

JESSEWARDEN.COM

Total Functions

✦ WHEN TO APPLY: FUNCTION-LEVEL

ENCHANTMENT | INTERMEDIATE

ANY HONDA

```
Cold? ✓ Starts
20yr old? ✓ Starts
Crashed? ✓ Starts
```

Total. Predictable. Every input handled.

✅ All Inputs | 🛡️ No Exceptions | ⚡ Intermediate

Function that returns a value for every possible input. No exceptions, no undefined behavior — predictable for every case.

```
divide(a: number, b: PositiveNumber): number
// impossible to pass zero
```

"Any Honda: Cold? Hot? 50 years old? Starts anyway."

LEARN MORE

Partial Functions

☠ ANTI-PATTERN OF: TOTAL FUNCTIONS

CURSE | HIDDEN CRASHES | AVOID

💀 Hidden Crashes ⚠ Edge Cases ☠ Avoid

Functions that throw or crash for some inputs — hidden failure modes not visible in the type signature.

```
function divide(a: number, b: number) {
return a / b
}
// returns Infinity or NaN 💥
```

"I added a New Relic dimension and it crashed my whole application!"

JESSEWARDEN.COM

Smart Constructors

✦ When to apply: domain modeling

Enchantment · Intermediate

DELIVERED BY AMAZON

Email.create(input)

Result<Email, InvalidEmailError>

Buy from a reputable source. Guaranteed.

Safe Factory · Result Return · Intermediate

Factory functions that validate and return Result or Option instead of throwing. Caller always knows if construction can fail.

```
const email = Email.create(input)
// Result<Email, InvalidEmailError>
```

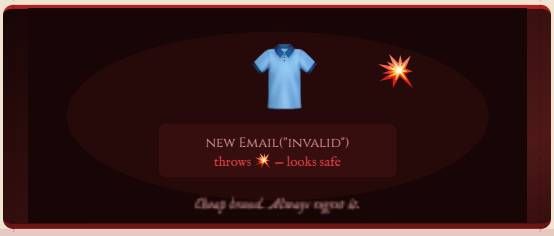
"Smart Constructors are like hewing inspectors — what's wrong, is it worth buying, and what to fix."

Learn more

Constructor Exceptions

☠ ANTI-PATTERN OF: SMART CONSTRUCTORS

CURSE | UNSAFE NEW | AVOID



💀 HIDDEN THROWS ⚠ UNSAFE NEW ☠ AVOID

Constructors that throw on invalid input — caller doesn't know from the type signature that it can fail.

```
new Email("invalid") // throws 💥
// but new Email(input) looks safe
// nothing in the signature warns you
```

"Whenever I buy from a non-reputable brand, I almost always regret it. Bad stitching. Falling apart."

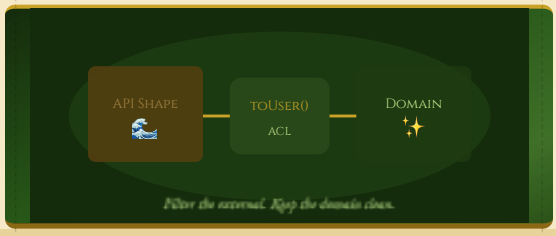
JESSEWARDEN.COM

Anti-Corruption Layer

★

✦ WHEN TO APPLY: BOUNDARIES/IO, SYSTEM-LEVEL

ENCHANTMENT | ADVANCED



🛡️ Clean Boundary ✅ Domain Safe 🔥 Advanced

Translate external/legacy types into your domain types at the boundary. Keep external shapes outside your core.

```
const user = toUser(apiResponse)
// API shape stays at the edge
// domain shape lives inside
```

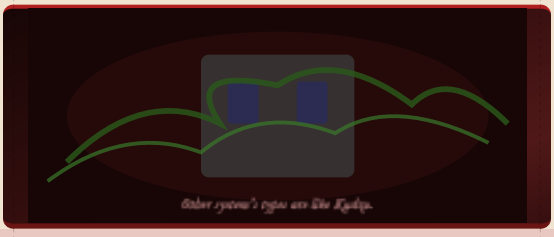
"I take my cousin's fashion advice, keep what fits me, discard what doesn't. Both valid in their context."

LEARN MORE

Leaky Abstractions

ANTI-PATTERN OF: ANTI-CORRUPTION LAYER

CURSE | API SPREAD | AVOID



💀 Tight Coupling ⚠ API Spread ☠ Avoid

External API shapes spreading through codebase, coupling your domain to third-party structures. Also leads to Shotgun Parsing.

```
function processUser(user: APIResponse)
// deep in business logic
// API changes → everything breaks 💥
```

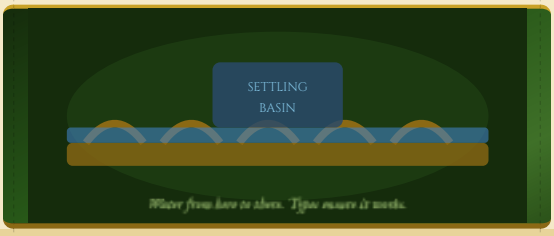
"Other system's types are like Kryptonite."

JESSEWARDEN.COM

Pipeline Builder

✦ When to apply: decoders, validation

Enchantment | Advanced



🔗 Collect All ✅ All Errors 🔥 Advanced

Pattern to build a function for processing data using a series of piped functions. Types ensure the pipeline connects safely.

```
Success()
.concat(isPasswordLongEnough(pw))
.concat(isPasswordStrongEnough(pw))
.map(_ ⇒ pw)
```

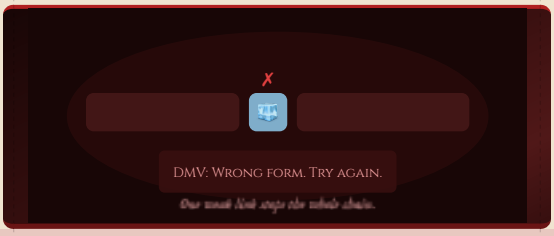
"Roman Aqueducts: settling basins slow water to safe speed. Arcades cross rivers. Siphons deliver on incline. Connected, they ensure delivery no matter the terrain."

Learn more

Fast-Fail Validation Chains

☠ ANTI-PATTERN OF: PIPELINE BUILDER

CURSE | STOPS EARLY | AVOID



💀 First Error Only ⚠ Stops Early ☠ Avoid

Imperative or monadic validation often stops at the first failure, so users only see one error at a time.

```
if (!name) return Left('name')
if (!email) return Left('email')
// only the first error returned
```

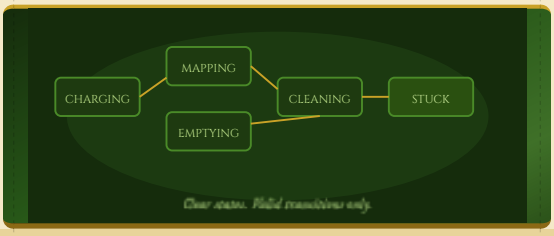
"A chain is only as strong as its weakest link. If all links are weak, it's not a strong chain."

JESSEWARDEN.COM

Typestate

✦ When to apply: state machines

Enchantment · Intermediate



🔒 Compile-Time ✅ Valid Transitions ⚡ Intermediate

Encode lifecycle states in types so only valid transitions are possible at compile time. Invalid transitions don't compile.

```
type Draft = { tag: 'Draft' }
type Submitted = { tag: 'Submitted' }
function submit(d: Draft): Submitted
```

"The Roomba: if full power, it cleans. If stuck, it turns around. If low power or full bin, it charges. Clear transitions only."

Learn more

Illegal State Transitions

☠ ANTI-PATTERN OF: TYPESTATE

CURSE | RUNTIME ONLY | AVOID

💀 Wrong Order ⚠ Runtime Only ☠ Avoid

Operations run in the wrong order because lifecycle rules are only enforced by runtime checks, not the type system.

```
approve(order)
// compiles even when order
// is still in Draft state 💥
```

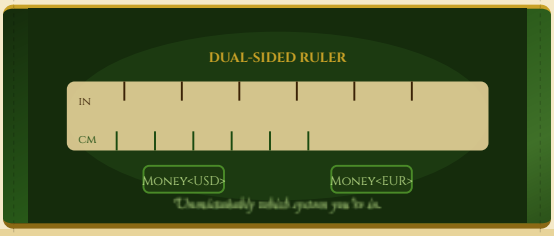
"Put food in the oven. 20 minutes later realized I forgot to turn it on."

JESSEWARDEN.COM

Indexed Types

✦ When to apply: domain modeling, value-level

Enchantment | Advanced



Type-Tagged ✅ Invariants 🔥 Advanced

Carry useful facts in type parameters (currency, units, state) so operations preserve invariants at compile time.

```
type Money<C> = { amount: number; currency: C }
add(usd1, usd2) // ok ✓
add(usd1, eur1) // compile error ✓
```

"Rulers: inches on one side, metric on the other. Unmistakably clear which system you're measuring in."

Learn more

Currency Mixups

☠ ANTI-PATTERN OF: INDEXED TYPES

CURSE | SILENT MIXUP | AVOID

💀 Silent Mixup ⚠ Wrong Total ☠ Avoid

Monetary values with different currencies treated as plain numbers — invalid operations compile and produce silently incorrect totals.

```
add({ amount: 10, currency: 'USD' },
{ amount: 7, currency: 'EUR' })
// silently mixes currencies 💥
```

"My daughter kept getting wrong-sized clothes from AliExpress — their size chart was for a completely different market."

JESSEWARDEN.COM

Units-of-Measure Types

★

✦ WHEN TO APPLY: VALUE-LEVEL

ENCHANTMENT | INTERMEDIATE

Meters ÷ Seconds = m/s

ADDMETERS(D, T) → COMPILE ERROR ✓

"Units enforce the math. Always."

🪚 Unit-Safe ✅ Dimension Check ⚡ Intermediate

Encode physical/business units in types to prevent invalid arithmetic across incompatible dimensions.

```
type Meters = number & { _unit: 'm' }
type Seconds = number & { _unit: 's' }
addMeters(distance, time) // compile error ✓
```

"How much wood could a woodchuck chuck, if a woodchuck could chuck wood?"

LEARN MORE

Unit Mixups

ANTI-PATTERN OF: UNITS-OF-MEASURE TYPES

CURSE | UNIT CHAOS | AVOID

💀 Unit Chaos ⚠ Silent Wrong ☠ Avoid

Values with different units treated as plain numbers and combined incorrectly — no compile-time protection against dimension errors.

```
const bad = distance + time
// Meters + Seconds = ???
// compiles. silently wrong. 💥
```

"Mars Climate Orbiter crashed because one team used metric, another used imperial. $327M mistake."

JESSEWARDEN.COM

Capability-Based Design

✦ When to apply: system-level, API design

Enchantment | Advanced

🎯 Explicit Caps ✅ No Surprises 🔥 Advanced

Pass explicit capabilities (permissions/operations) into functions instead of relying on ambient global authority.

```
function registerUser(
sendEmail: SendEmail,
user: User
) { ... }
```

"You get a glimpse at someone's wardrobe looking at the clothes drying on the clothesline."

Learn more

Ambient Authority

ANTI-PATTERN OF: CAPABILITY-BASED DESIGN

CURSE | HIDDEN EFFECTS | AVOID

Hidden Effects ⚠ Global Reach ☠ Avoid

Code can perform hidden side effects because it can reach global DB/fs/network clients directly.

```
// function that should only email users
// also mutates billing records 💥
// caller had no idea
```

"Turned on a light switch and a fan started, a click turned on, and something beeped."

JESSEWARDEN.COM

Type Proofs (Assertion Functions)

◆ WHEN TO APPLY: VALUE-LEVEL, FUNCTION-LEVEL, DOMAIN MODELING

ENCHANTMENT | ADVANCED

PROOF

```
Given: p[0] < p[1]
Assert: assertOrdered(p)
∴ p is OrderedPair ✓
// compiler now knows!
```

Show your work. Test that work.

🔬 Proven ✅ Compiler Knows 🔥 Advanced

Use assertion functions to prove runtime predicates to the type system, enabling refined types after checks and composed proofs.

```
function assertOrdered(
p: readonly [number, number]
): asserts p is OrderedPair {
if (p[0] ≥ p[1]) throw new Error('Expected a <
b')
}
```

"When you do the math, show your work, and test that work."

LEARN MORE

Unproven Invariants

☠ ANTI-PATTERN OF: TYPE PROOFS (ASSERTION FUNCTIONS)

CURSE | UNSAFE CASTS | AVOID

TRUSTED BY... COMMENT

```
// assume sorted
tuple[0] + tuple[1]
// oops, reordered above 💥
range check: BROKEN
```

The types aren't total.

💀 Assumed Only ⚠ Comment-Only ☠ Avoid

Code assumes ordering or constraints that are only documented in comments, forcing unsafe casts and brittle logic.

```
// assume sorted — trust me bro
const [low, high] = tuple
// later: tuple reordered above
// range check: silently broken 💥
```

"The types aren't total."

JESSEWARDEN.COM

Error Algebra — Errors as Values

◆ WHEN TO APPLY: API DESIGN, SYSTEM-LEVEL

ENCHANTMENT | INTERMEDIATE

CHECKOUT ERROR ALGEBRA

`CardDeclined`	→ RETRY CARD
`InventoryChanged`	→ REFRESH
`TransientNetwork`	→ BACKOFF

Named. Handled. Recoverable.

🎯 Named Errors ✅ Recovery Path ⚡ Intermediate

Model domain errors as discriminated unions with explicit recovery meaning: retry, fix input, escalate, or abort.

```
type CheckoutError =
| { tag: 'CardDeclined' }
| { tag: 'InventoryChanged' }
| { tag: 'TransientNetwork'
retryAfterMs: number }
```

"We know what is wrong, how to handle it, and some things we can try to recover."

LEARN MORE

Stringly-Typed Failures

☠ ANTI-PATTERN OF: ERROR ALGEBRA — ERRORS AS VALUES

CURSE | OPAQUE ERRORS | AVOID

💀 Opaque Errors ⚠ Guess Recovery ☠ Avoid

Generic errors hide cause and recovery strategy, so callers are left guessing what to do next.

```
throw new Error('checkout failed')
// why? retry? fix input?
// escalate? no clue. 🎲
```

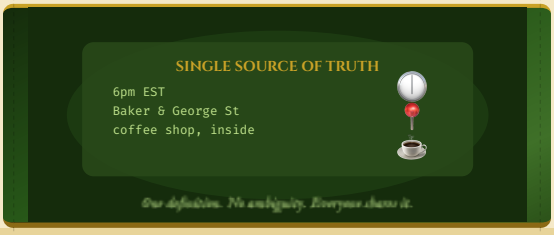
"Car won't start. No lights on the dash. Something broke — no idea why, or what NOT to do."

📜 JESSEWARDEN.COM

Schema / Codec Single Source of Truth

◆ WHEN TO APPLY: BOUNDARIES/IO, MODULE-LEVEL

ENCHANTMENT | BEGINNER



📋 One Schema ✅ Auto-Derived ✦ Beginner

Define runtime schema once and derive parser, encoder, and static types from it to avoid drift between sources.

```
const UserSchema = z.object({
id: z.string(),
email: z.string().email()
})
type User = z.infer<typeof UserSchema>
```

"Meet me at 6pm EST inside the coffee shop on Baker & George St. One definition. No ambiguity."

LEARN MORE

Schema Drift

☠ ANTI-PATTERN OF: SCHEMA / CODEC SINGLE SOURCE OF TRUTH

CURSE | TYPE DRIFT | AVOID

💀 Type Drift ⚠ Silent Bugs ☠ Avoid

Runtime validators, TypeScript types, and API docs diverge over time and create subtle bugs nobody sees coming.

```
// TS type says: email?: string
// Parser requires: email (required)
// API docs say: e_mail
// Which is right? 🤷
```

"You said it!" "Yeah, you, not me." Basic miscommunication. From two separate sources of truth."

JESSEWARDEN.COM

Property-Based Testing from Types

◆ WHEN TO APPLY: FUNCTION-LEVEL, DOMAIN MODELING

ENCHANTMENT | INTERMEDIATE

QA ENGINEER WALKS INTO A BAR

```
orders: 1 beer ✓
orders: 0 beers ✓
orders: 999999999 ✓
orders: a lizard ✓
orders: -1 beers ... 🤔
```

Every possible input. Every time.

🎲 100+ Cases ✅ Invariants ⚡ Intermediate

Generate many test cases from typed generators and assert invariants instead of relying only on hand-picked examples.

```
fc.assert(
fc.property(
arbUser,
u ⇒ deepEqual(decode(encode(u)), u)
)
)
```

"A QA Engineer walks into a bar. Orders 0 beers. Orders 99999999. Orders a lizard. Orders -1."

LEARN MORE

Example Blind Spots

ANTI-PATTERN OF: PROPERTY-BASED TESTING FROM TYPES

CURSE | EDGE CASE MISS | AVOID

Edge Case Miss | Happy Path Only | Avoid

Small example-based tests miss edge cases that violate invariants in production — things you never thought to test.

```
// Tests pass for ASCII names
// but fail for Unicode input
// never thought to test that
```

"Deadpool: Shoot... did I leave the stove on!"

JESSEWARDEN.COM

Option / Maybe Type

✦ WHEN TO APPLY: DOMAIN MODELING, FUNCTION-LEVEL

ENCHANTMENT | BEGINNER

📦 SOME | NONE ✅ FORCED HANDLE ✦ BEGINNER

Represent absence explicitly with Some/None instead of null or undefined. Forces callers to handle the Nothing scenario.

```
type Option<T> =
| { tag: 'Some', value: T }
| { tag: 'None' }
// map and getWithDefault helpers
```

"Do you have pork basil today? Maybe they do. If not: popcorn chicken. Absence is handled gracefully."

LEARN MORE

Null Pointer Branching

☠ ANTI-PATTERN OF: OPTION / MAYBE TYPE

CURSE | NULL CRASH | AVOID

💀 Null Crash ⚠ Forgotten Check ☠ Avoid

null/undefined checks spread across code and are easy to forget in one branch — one missed check crashes everything.

```
user.address.city
// throws when address is undefined
// forgot to check in this branch 💥
// user?.address?.city just moves the problem
```

"Agent Smith: What good is a phone call if you are unable to speak?"

JESSEWARDEN.COM

Product Types (Records)

◆ When to Apply: Domain Modeling

ENCHANTMENT | BEGINNER

BLUEPRINT: MONEY

```
amount: number
currency: Currency
```

BREED + NAME + AGE = DOG
AMOUNT + CURRENCY = MONEY

Blueprint of everything. AND composition.

📋 Named Fields ✅ AND Together ✦ Beginner

Model data that must exist together with named fields (AND composition). The blueprint of everything.

```
type Money = {
amount: number
currency: Currency
}
```

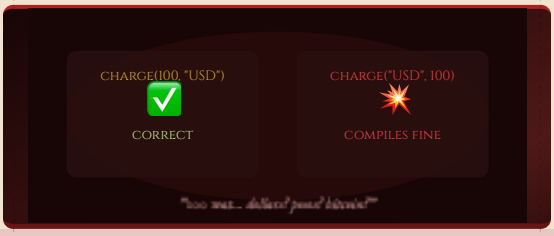
"We're all made of many parts — a name, where we live, our interests. A Record holds it all together."

LEARN MORE

Positional and Parallel Data Bugs

☠ ANTI-PATTERN OF: PRODUCT TYPES (RECORDS)

CURSE | POSITIONAL BUGS | AVOID



💀 Positional Bugs ⚠ Order Matters ☠ Avoid

Loosely related values passed separately, reordered accidentally, or updated inconsistently — the compiler can't help.

```
charge(100, 'USD') // correct ✓
charge('USD', 100) // compiles 💥
// both are valid JS — wrong result
```

"100 what… dollars? pounds? bitcoin? Something broke — where's the error and how do I fix it?"

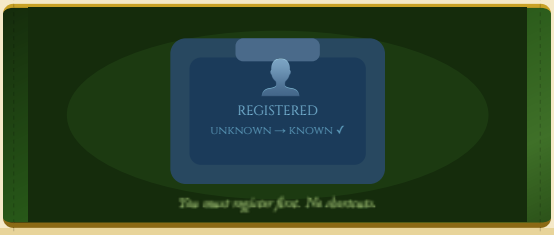
JESSEWARDEN.COM

Unknown at Boundaries

★

◆ WHEN TO APPLY: BOUNDARIES/IO

ENCHANTMENT | BEGINNER



🔒 UNKNOWN FIRST ✅ PARSE ONCE ✦ BEGINNER

Accept external input as unknown, then parse/narrow once into trusted domain types. Enforces the conversion.

```
function parseUser(
input: unknown
): Result<User, ParseError>
```

"Guests get a temporary badge, but you must register them first. unknown enforces that conversion."

LEARN MORE

Any-Typed Boundary Leaks

☠ ANTI-PATTERN OF: UNKNOWN AT BOUNDARIES

CURSE | UNSAFE ANY | AVOID

💀 Unsafe any ⚠ Late Failures ☠ Avoid

Untrusted inputs enter the core system without parsing, causing late runtime failures deep in business logic.

```
// API payload typed as any
// reaches business logic
// crashes on missing fields 💥
```

"Oh yeah, this random dude I found on the street is with me." — Not safe. Really not safe."

JESSEWARDEN.COM

Readonly by Default

★

✦ When to apply: module-level, function-level

Enchantment | Intermediate

🔒 Readonly ✅ Explicit Change ⚡ Intermediate

Prefer immutable shapes so mutation is explicit and local, reducing aliasing bugs and spooky action at a distance.

```
type User = {
readonly id: UserId
readonly email: Email
}
```

"You don't make up and '1' suddenly equals '2'. Immutability is how the universe works — predictable."

Learn more

Incidental Mutation

☠ ANTI-PATTERN OF: READONLY BY DEFAULT

CURSE | ALIASING BUGS | AVOID

💀 Aliasing Bugs ⚠ Spooky Action ☠ Avoid

Shared object references mutated unexpectedly, causing non-local bugs and flaky behavior nobody can trace.

```
// helper function mutates user.roles
// breaks permission checks elsewhere
// nobody saw it coming ☠
```

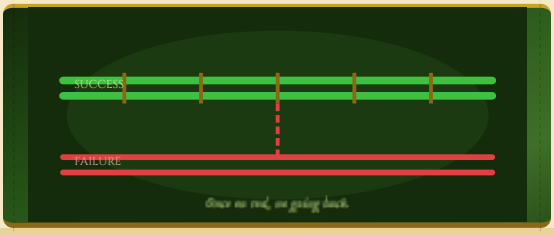
"CounterStrike: gown in and the camera moved through the wall. Mutation = danger."

JESSEWARDEN.COM

Railway-Oriented Programming

◆ WHEN TO APPLY: FUNCTION-LEVEL, API DESIGN

ENCHANTMENT | INTERMEDIATE



🚂 Two Tracks ✅ Composable ⚡ Intermediate

Compose Result/Maybe steps left-to-right — stop at first failure, continue on success. Two tracks: green and red.

```
parse(input)
.andThen(validate)
.andThen(save)
.andThen(sendWelcomeEmail)
```

"Expect the worst, hope for the best."

LEARN MORE

Nested Error Handling

☠

ANTI-PATTERN OF: RAILWAY-ORIENTED PROGRAMMING

CURSE | NESTED HELL | AVOID

```
if (ok1) {
   if (ok2) {
      try {
         ...
      } catch { ... }
   } else { ... }
}
```

So many rocks. No one will rescue you.

💀 Nested Hell ⚠ Hard to Read ☠ Avoid

if/else + try/catch chains become hard to read and easy to break as the number of steps grows.

```
if (ok1) {
if (ok2) {
try { ... } catch { ... }
} else { ... }
} else { ... }
```

"So many submerged rocks in a frothing sea, near a lighthouse. No one will rescue you."

JESSEWARDEN.COM